

3 Value types

3.1 Structs

In Java, the type system distinguishes between primitive types and objects. For example, the memory semantics of the types

```
int x = 0;
```

`Point p = new Point<int, int>(0, 0)`(where `Point` is a typical tuple of ints) are different in two ways.

1. `int` has value semantics whereas `p` has reference semantics. Imagine we have the following two methods that operate over ints and Points.

```
public static void changeInt(int x) { x = 5; }
```

```
public static void changePoint(Point p) { p.X = 5; }
```

If we call `changeInt(x)` and then inspect the value of `x` afterwards, we will find that the value of `x` is 0. This is because we pass `x` by-value, that is we pass a copy of `x`, to `changeInt`. If we call `changePoint(p)` and then inspect `p.x`, we'll see that `p.x` is now 5. This is because we pass `p` by-reference, that is we pass a reference of `p`, to `changePoint`. Thus we are actually modifying `p` vs. a copy of it.

2. `x` is allocated on the stack and contains exactly the bits of the integer whereas `p` is allocated on the heap and contains a reference to that memory in the heap. Also, the memory allocated on the heap contains other data other than the components of the pair, e.g., a reference to the vtable of `Point` class.

Furthermore, since `x` is allocated on the stack, its memory is "released" when the method that allocated `x` is returns. Since `p` is allocated on the heap, its memory is not released until the garbage collector cleans up.

The distinction is important in understanding the costs involved with allocating lots of small objects. For example, we may allocate lots of `Point` objects in doing GUI or graphics that will require many allocations on the heap. Furthermore, these short-lived objects will need to be garbage collected eventually which will incur more cost.

Ideally, we would like to treat `Point` as a primitive type where we avoid heap allocations, but Java does not provide any way to do this have this facility. C# allows us to create such types, called value types, via the `struct` construct.

```
public struct Vec2D  
{
```